

Phantom

Autonomous Adversary Simulation Platform

System Documentation & Technical Guide

Version 0.9.38

March 2026

Author: Gadouri Rodwan

Supervisor: Dr. Allama Oussama

Repository: <https://github.com/Usta0x001/Phantom>

License: Apache 2.0

Abstract

Phantom is an AI-powered autonomous penetration testing platform that leverages Large Language Models (LLMs) to discover, exploit, and verify real-world vulnerabilities in web applications, APIs, and network services — entirely without human intervention. Unlike signature-based scanners that match known patterns, Phantom employs a **ReAct** (Reason + Act) agent loop that reads HTTP responses, reasons about attack surfaces, chains multi-step exploits, and produces working proof-of-concept code.

This document serves as the **complete technical documentation and user guide** for the Phantom system. It covers architecture, design philosophy, implementation details, security controls, scanning workflows, LLM integration, the full tool ecosystem, quality assurance, and deployment instructions.

Key result: Against the OWASP Juice Shop benchmark, Phantom v0.9.38 discovered 7 vulnerabilities (5 Critical, 2 High) in 26 iterations over 15 minutes at a cost of \$1.0.

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Objectives	4
1.3	Scope	4
1.4	At a Glance (v0.9.38)	5
2	System Architecture	5
2.1	Design Philosophy	5
2.2	High-Level Architecture	5
2.3	Package Structure	6
3	Agent Architecture	6
3.1	The ReAct Loop	6
3.2	Stop Conditions	7
3.3	Approaching-Max Warning	7
3.4	Agent State	8
3.5	System Prompt	8
3.5.1	Vulnerability Focus Areas	8
4	LLM Integration	8
4.1	Client Architecture	8
4.2	Memory Compression	9
4.3	Provider Registry	9
4.4	Fallback Chain	9
5	Tool Ecosystem	9
5.1	Registration & Execution	9
5.2	Complete Tool Inventory	10
5.3	Auto-Recording Pipeline	11
6	Security Controls	11
6.1	Defence-in-Depth Model	11
6.2	Scope Validator	12
6.3	Audit Logger	12
6.4	Cost Controller	13
6.5	Verification Engine	13
7	Scanning Workflow	14
7.1	Scan Lifecycle	14
7.2	Post-Scan Enrichment Pipeline	14
7.3	Scan Profiles	15
7.4	Scan Resume	15
8	Configuration & Deployment	15
8.1	Environment Variables	15
8.2	Quick Start	15
8.3	Credential Storage	16
8.4	Docker Sandbox	16
8.5	Operating Modes	16
9	Quality Assurance	16

9.1	Test Suite	16
9.2	Audit History	17
9.3	Benchmark Results	17
10	LLM Selection Guide	17
10.1	Requirements	17
10.2	Model Comparison	17
10.3	Recommendations	18
11	Recommendations & Future Work	19
11.1	System Quality Scores	19
11.2	Predictions	19
11.3	Short-Term Roadmap (v0.9.39–v0.9.41)	19
11.4	Medium-Term (v0.10.x)	20
11.5	Long-Term (v1.0)	20
11.6	Operator Guidelines	20
12	Conclusion	20
A	Glossary	21
B	Full Configuration Reference	21

1 Introduction

1.1 Motivation

Traditional penetration testing is expensive, time-consuming, and suffers from a critical talent shortage. According to (ISC)², the global cybersecurity workforce gap exceeds 3.4 million professionals. Automated scanners like Nessus, Burp Suite, and OWASP ZAP address only known vulnerability signatures, missing business logic flaws, chained exploits, and novel attack vectors that require human-like reasoning.

Phantom bridges this gap by combining the reasoning capabilities of frontier LLMs with a rich toolkit of offensive security instruments, all operating within a sandboxed Docker environment. The system autonomously:

1. Reconnoitres the target (port scanning, web crawling, technology fingerprinting)
2. Discovers attack surfaces (endpoints, parameters, authentication flows)
3. Exploits vulnerabilities with working PoC payloads
4. Verifies findings through multiple confirmation strategies
5. Generates structured reports with CVSS scoring, MITRE ATT&CK mappings, and compliance mapping

1.2 Objectives

- Design and implement an autonomous adversary simulation agent
- Achieve parity with or exceed manual penetration testing for common web vulnerabilities
- Ensure safe operation through defence-in-depth (sandboxing, scope validation, cost control)
- Provide actionable, standards-compliant vulnerability reports
- Support multiple LLM providers for flexibility and cost optimisation

1.3 Scope

Phantom targets:

- Web applications (black-box and grey-box)
- REST / GraphQL APIs
- Network services (via Nmap)
- Source code repositories (static analysis via Semgrep)

1.4 At a Glance (v0.9.38)

Table 1: Phantom v0.9.38 metrics

Metric	Value
Python source files	124
Lines of Python code	~31,900
Registered security tools	55
Scan profiles	5
Test cases (passing)	731
Test cases (skipped)	97
System prompt length	407 lines
Defence layers	11
LLM providers supported	7+
License	Apache 2.0
Minimum Python version	3.12

2 System Architecture

2.1 Design Philosophy

Phantom is built on four core principles:

1. **LLM-guided reasoning over signature matching** — The agent reads responses, identifies attack surfaces, and chains exploits using natural language understanding rather than pattern databases.
2. **Verify before report** — Every finding must have a working PoC. Unverified findings are flagged but never falsely promoted.
3. **Sandbox-first execution** — All offensive tools run inside ephemeral Docker containers with no host network access, no privilege escalation, and restricted capabilities.
4. **Never lose data** — The findings ledger survives memory compression. HMAC-chained audit trails use `fsync` for crash safety.

2.2 High-Level Architecture

Figure 1 shows how Phantom’s components interact. The user (or CI pipeline) provides a target through the CLI. The **PhantomAgent** orchestrates everything: it queries the LLM for reasoning, dispatches tool calls through the **Tool Executor**, and tracks all state. Offensive tools run inside an ephemeral **Docker Sandbox** (Kali Linux) for isolation. Security controls and the post-scan enrichment pipeline operate on the host side.

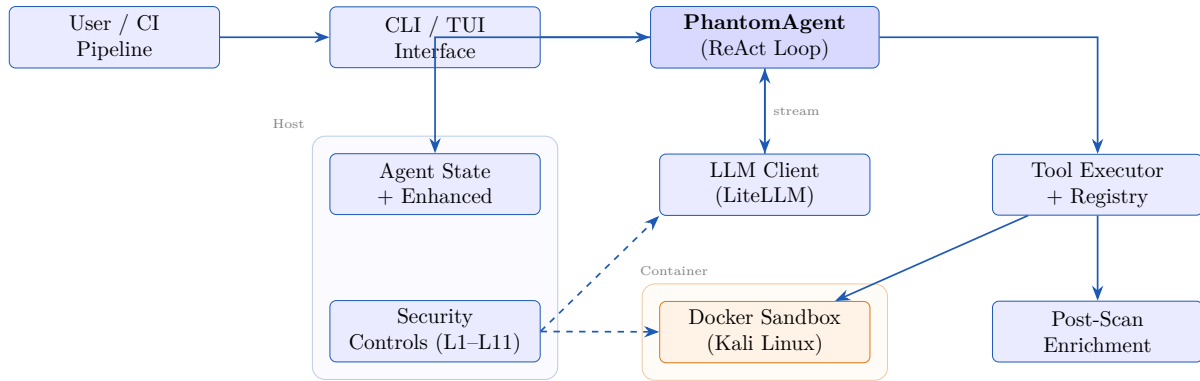


Figure 1: Phantom high-level architecture. Solid arrows show data flow; dashed arrows show security enforcement boundaries.

2.3 Package Structure

Table 2: Phantom package organisation

Package	Purpose
<code>phantom.agents</code>	Agent core: ReAct loop, state machine, PhantomAgent persona
<code>phantom.core</code>	Security & domain logic: 17 modules including scope validation, verification, audit logging
<code>phantom.tools</code>	Tool implementations: 15 subdirectories, 55 registered tools
<code>phantom.llm</code>	LLM integration: client, provider registry, memory compressor, config
<code>phantom.runtime</code>	Docker sandbox lifecycle and HTTP tool-server communication
<code>phantom.models</code>	Pydantic domain models: Vulnerability, Host, ScanResult, ScanProfile
<code>phantom.skills</code>	Markdown knowledge base: 9 vulnerability categories loaded into system prompt
<code>phantom.interface</code>	CLI (Typer) and TUI (Rich) user interfaces
<code>phantom.telemetry</code>	Observability: tracer, vulnerability report aggregation
<code>phantom.config</code>	Configuration management with OS keyring support
<code>phantom.utils</code>	Shared utilities

3 Agent Architecture

3.1 The ReAct Loop

Phantom's core engine implements a **Reason + Act** (ReAct) loop, a paradigm where the agent alternates between thinking about observations and taking actions via tool calls. The implementation resides in `base_agent.py` (824 lines).

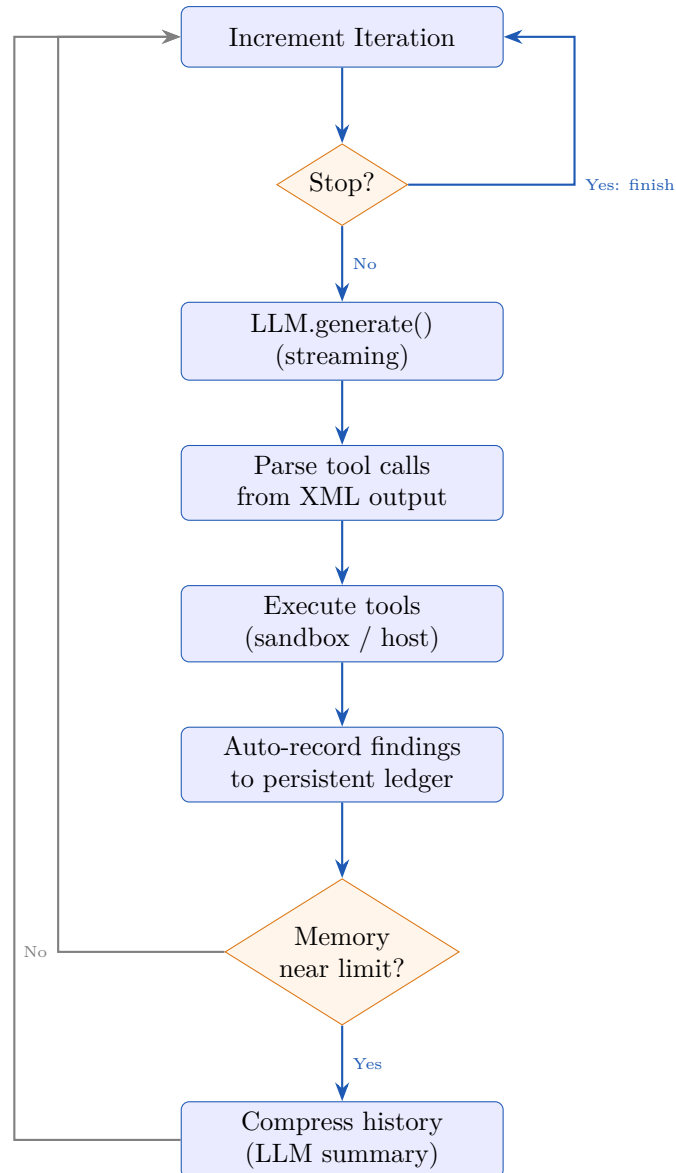


Figure 2: The ReAct loop workflow. Each iteration queries the LLM, executes the requested tools, records findings, and optionally compresses conversation history when approaching the context window limit.

3.2 Stop Conditions

The agent loop terminates when any of these conditions is met:

- **Iteration cap:** $\text{iteration} \geq \text{max_iterations}$ (default: 300)
- **Completion:** Agent calls `finish_scan` tool
- **User abort:** `stop_requested` flag (Ctrl+C)
- **Cost limit:** Budget exceeded (\$25 default)
- **LLM failure:** After exhausting all retries

3.3 Approaching-Max Warning

At 85% of `max_iterations` (configurable threshold), the agent receives a system warning to prioritise high-value findings. At `max-3` iterations, a final wrap-up message is injected (guarded: only fires when `max > 3`).

3.4 Agent State

`AgentState` (268 lines) is a Pydantic model tracking:

- Conversation history (max 500 messages)
- Findings ledger (immune to memory compression)
- Unverified findings queue
- Actions taken and observations (max 5,000 each)
- Iteration counter, phase, timing, cost accumulation

`EnhancedAgentState` (553 lines) extends this with:

- Vulnerability tracking (`dict[str, Vulnerability]`)
- Host/service enumeration
- Endpoint tracking with deduplication (capped at 10,000)
- CVSS-based priority queues
- Checkpoint save/restore for scan resume
- Knowledge store integration for false-positive filtering

3.5 System Prompt

The system prompt is a 407-line Jinja2 template defining the agent's persona, capabilities, communication rules, and vulnerability focus areas. It dynamically loads security skill documents based on the target type.

3.5.1 Vulnerability Focus Areas

1. IDOR (Insecure Direct Object References)
2. SQL Injection
3. Server-Side Request Forgery (SSRF)
4. Cross-Site Scripting (XSS)
5. XML External Entity (XXE)
6. Remote Code Execution (RCE)
7. Cross-Site Request Forgery (CSRF)
8. Race Conditions
9. Business Logic Flaws
10. Authentication & JWT Weaknesses

4 LLM Integration

4.1 Client Architecture

The LLM client (`llm.py`, 454 lines) is built on **LiteLLM**, providing a unified interface to 100+ LLM providers. Key features:

- **Streaming:** Processes responses chunk-by-chunk, stops at `</function>` tag
- **Dynamic context window:** Reads model's actual context window from provider registry; sets compression threshold at 75%
- **Retry with exponential backoff:** Up to 5 retries; wait = $\min(10, 2 \cdot 2^n) + \text{jitter}$
- **Cost tracking:** Per-request `RequestStats` with input/output/cached tokens and USD cost
- **Temperature:** None (uses model default) per v0.9.34

4.2 Memory Compression

When conversation history approaches the context window limit (75% threshold), the `MemoryCompressor` (448 lines) summarises older messages using a specialised 10-rule prompt that preserves:

- Exact URLs, ports, endpoints, IP addresses
- Exact payloads, HTTP headers, cookies, credentials
- Full technology stack details
- All failed attempts (to avoid repetition)
- Attack surface maps and open ports

The findings ledger is **never compressed** — it is injected into summaries but maintained separately as a persistent record.

4.3 Provider Registry

Phantom ships with 18+ pre-configured model presets across 7 providers:

Table 3: Pre-configured LLM providers

Provider	Models
Groq	llama-3.3-70b-versatile, llama-3.1-8b-instant, llama3-70b-8192
OpenAI	gpt-4o (128K ctx), gpt-4o-mini
Anthropic	claude-sonnet-4-20250514 (200K ctx), claude-3-5-haiku-latest
Google	gemini-2.5-flash (1M ctx)
Ollama	llama3:70b (local, free)
OpenRouter	deepseek-v3.2, deepseek-chat-v3-0324, gemini-2.5-flash, llama-3.3-70b-instruct, gemma-3-27b-it:free, hermes-3-llama-3.1-405b:free, qwen3-coder:free, mistral-small-3.1-24b:free, grok-4.1-fast

4.4 Fallback Chain

The `FallbackChain` class provides automatic model failover. If the primary model fails (rate limit, quota, outage), it advances to the next configured provider, logging the transition and tracking failure counts.

5 Tool Ecosystem

5.1 Registration & Execution

Tools are registered via the `@register_tool` decorator with XML schema files for parameter validation. The executor dispatches tools either to the Docker sandbox (via HTTP) or locally, depending on the `sandbox_execution` flag.

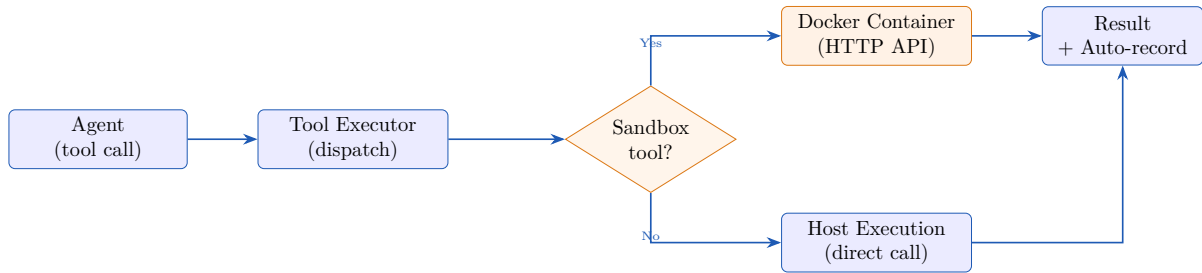


Figure 3: Tool execution dispatch: sandbox tools run in the Docker container via HTTP; host-side tools (thinking, reporting, agent management) run locally.

5.2 Complete Tool Inventory

Table 4: All 55 registered tools

Category	Tool	Purpose
Thinking	<code>think</code>	Chain-of-thought reasoning
Nmap	<code>nmap_scan</code>	Port scanning + service detection
	<code>nmap_vuln_scan</code>	Vulnerability script scanning
Nuclei	<code>nuclei_scan</code>	Template-based vulnerability scanning
	<code>nuclei_scan_cves</code>	Known CVE detection
	<code>nuclei_scan_misconfigs</code>	Misconfiguration detection
SQLMap	<code>sqlmap_test</code>	SQL injection detection
	<code>sqlmap_forms</code>	Form-based SQLi testing
	<code>sqlmap_dump_database</code>	Database extraction
FFuf	<code>ffuf_directory_scan</code>	Directory brute-forcing
	<code>ffuf_parameter_fuzz</code>	Parameter discovery
	<code>ffuf_vhost_fuzz</code>	Virtual host enumeration
Httpx	<code>httpx_probe</code>	HTTP probing + tech fingerprinting
	<code>httpx_screenshot</code>	Screenshot capture
	<code>httpx_full_analysis</code>	Deep HTTP analysis
Katana	<code>katana_crawl</code>	Advanced web crawling
Subfinder	<code>subfinder_enumerate</code>	Subdomain enumeration
	<code>subfinder_with_sources</code>	Subfinder with source tracking
Verification	<code>verify_vulnerability</code>	Auto-verify via 8 strategies
	<code>check_known_vulns</code>	Query knowledge store
	<code>enrich_vulnerability</code>	MITRE ATT&CK + CWE mapping
Proxy	<code>send_request</code>	Send HTTP request with auth injection
	<code>repeat_request</code>	Replay captured request
	<code>list_requests</code>	List captured traffic
	<code>view_request</code>	View request/response details
	<code>scope_rules</code>	Manage scope rules
Terminal	<code>terminal_execute</code>	Execute shell commands in sandbox
Python	<code>python_action</code>	Run Python scripts in sandbox
Browser	<code>browser_action</code>	Playwright browser automation
Reporting	<code>create_vulnerability_report</code>	Detailed finding report (16+ params)

Table 4 — continued

Category	Tool	Purpose
	<code>report_vulnerability</code>	Quick finding report (5 params)
Finish	<code>finish_scan</code>	Finalise scan + enrichment pipeline
Agents	<code>create_agent</code>	Spawn specialised sub-agent
	<code>send_message_to_agent</code>	Inter-agent messaging
	<code>agent_finish</code>	Sub-agent completion

5.3 Auto-Recording Pipeline

The executor automatically extracts and records findings from tool outputs:

- **Nuclei:** severity, template name, matched URL
- **Nmap:** open ports, services, OS fingerprinting
- **Katana:** discovered URLs, API endpoints, forms
- **SQLMap:** injection points, vulnerability confirmation
- **send_request:** SQLi indicators, XSS reflections, sensitive data exposure

These findings survive memory compression and ensure critical discoveries are never lost, even if the LLM forgets about them during long scans.

6 Security Controls

6.1 Defence-in-Depth Model

Phantom implements 11 independent security layers. Each layer provides protection even if other layers are bypassed.

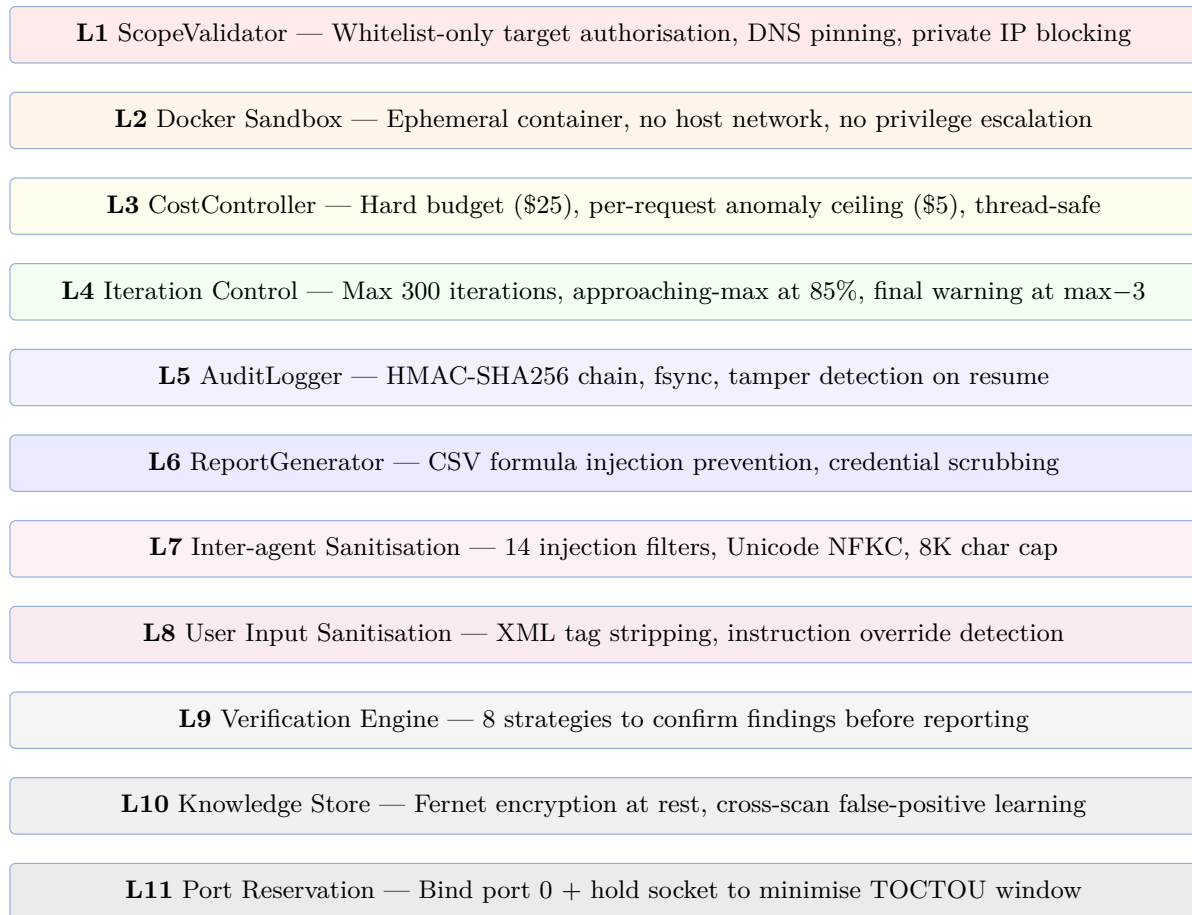


Figure 4: Phantom’s 11-layer defence-in-depth model. Each layer provides independent protection.

6.2 Scope Validator

The **ScopeValidator** (370 lines) enforces target authorisation:

- 4 rule types: domain (with wildcards), IP, CIDR, regex
- Deny rules override allow rules
- DNS pin cache (max 10,000 entries) prevents rebinding attacks
- ReDoS protection: pattern cap 500 chars, target cap 2,048 chars
- Private IP blocking prevents SSRF to internal networks

6.3 Audit Logger

The **AuditLogger** (449 lines) creates a tamper-evident, crash-safe audit trail:

- JSONL format, one JSON object per line
- Each entry contains `_prev_hash` and `_hmac` (HMAC-SHA256)
- Unique per-run key (`secrets.token_hex(32)`), protected with `chmod 0o600`
- `os.fsync` after every write for crash safety
- Auto-rotation at 50 MB
- Chain verification on resume; logs tampering detection events

6.4 Cost Controller

Table 5: Cost controller limits (all thread-safe with mutex lock)

Limit	Default	Purpose
Total scan cost	\$25.00	Hard budget cap
Per-request cost	\$5.00	Anomaly detection
Input tokens	5,000,000	Token ceiling
Output tokens	500,000	Token ceiling
Compression calls	50	Prevent compression spirals
Warning threshold	80%	Early alert

6.5 Verification Engine

The `VerificationEngine` (602 lines) implements 8 confirmation strategies:

1. **Time-based:** SQLi SLEEP, RCE sleep with 4.5s threshold
2. **Error-based:** SQL error pattern matching
3. **Boolean-based:** Boolean injection comparison
4. **DOM reflection:** XSS DOM reflection check
5. **OOB HTTP:** Out-of-band callback via `InteractshClient`
6. **OOB DNS:** Out-of-band DNS callback
7. **Known file:** LFI verification via known file reads
8. **Math eval:** SSTI math expression evaluation

Important

Failure to auto-verify does **not** mean the finding is a false positive. Unverified HIGH/-CRITICAL findings are flagged as [UNVERIFIED] but never discarded.

7 Scanning Workflow

7.1 Scan Lifecycle

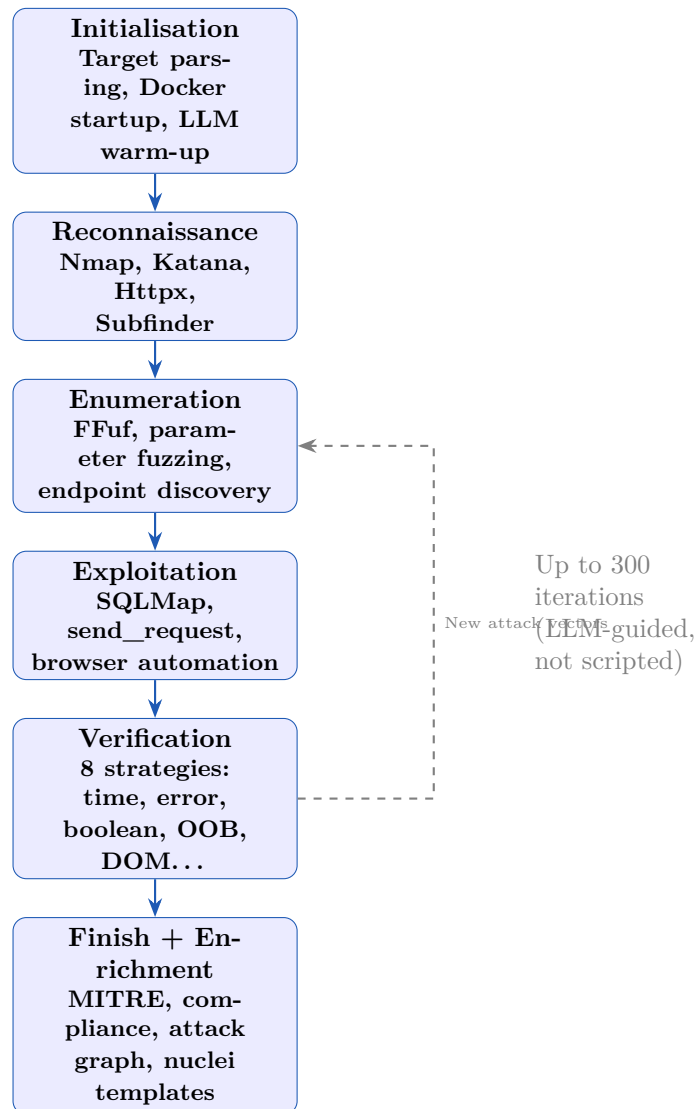


Figure 5: Scan lifecycle phases. The agent autonomously decides when to transition between phases based on its observations. The dashed loop-back arrow shows how new attack vectors discovered during verification can trigger additional enumeration.

7.2 Post-Scan Enrichment Pipeline

When `finish_scan` is called (minimum: 10 iterations + 8 tool calls), a 6-stage enrichment pipeline executes:

1. **Verification** — Re-exploits each finding via `VerificationEngine`
2. **MITRE Enrichment** — CWE, CAPEC, OWASP Top 10 mappings per finding
3. **Compliance Mapping** — OWASP/CWE/NIST/PCI DSS → `compliance_report.md`
4. **Attack Graph** — NetworkX graph → `attack_graph.json` + `attack_paths.md`
5. **Nuclei Templates** — YAML templates for each finding → `nuclei_templates/`
6. **Knowledge Store** — Persists vulns, hosts, history for cross-scan learning

7.3 Scan Profiles

Table 6: Available scan profiles

Profile	Iterations	Timeout	Memory	Browser	Special
Quick	300	120s	120K	Yes	Skip subfinder
Standard	120	120s	80K	Yes	Balanced
Deep	300	180s	100K	Yes	All tools, sub-agents
Stealth	60	60s	60K	No	2s delay, rate limit
API Only	100	120s	80K	No	No browser/subfinder

7.4 Scan Resume

Phantom checkpoints state every 10 iterations to `checkpoint.json`, including iteration count, `max_iterations`, phase, discovered hosts, endpoints, vulnerabilities, and the findings ledger. Crashed scans can be resumed:

```
1 phantom scan --resume <run-name>
```

8 Configuration & Deployment

8.1 Environment Variables

Table 7: Configuration environment variables

Variable	Default	Description
PHANTOM_LLM	(required)	LLM model identifier (e.g. <code>openrouter/deepseek/deepseek-v3.2</code>)
LLM_API_KEY	—	API key for LLM provider
PHANTOM_LLM_FALLBACK	—	Comma-separated fallback models
PHANTOM_MAX_COST_USD	25.0	Max LLM spend per scan
PHANTOM_LLM_MAX_RETRIES	5	Max retries per LLM request
LLM_TIMEOUT	300s	LLM request timeout
PHANTOM_REASONING_EFFORT	high	LLM reasoning effort level
PHANTOM_SANDBOX_IMAGE	ghcr.io/..	Docker sandbox image
PHANTOM_SANDBOX_EXECUTION_TIMEOUT	300s	Per-tool execution timeout
PHANTOM_DISABLE_BROWSER	false	Disable Playwright tools
PHANTOM_KNOWLEDGE_KEY	—	Fernet key for knowledge store
PHANTOM_LOG_LEVEL	INFO	Logging verbosity

8.2 Quick Start

```
1 # 1. Install Phantom
2 pip install phantom-agent
3
4 # 2. Configure LLM (cheapest recommended option)
5 export PHANTOM_LLM="openrouter/deepseek/deepseek-v3.2"
6 export LLM_API_KEY="sk-or-v1-your-key-here"
7
8 # 3. Pull sandbox image
9 docker pull ghcr.io/usta0x001/phantom-sandbox:latest
10
11 # 4. Run a scan
```

```
12 phantom scan -t http://target.example.com -m quick
```

Listing 1: Minimal setup to run Phantom

8.3 Credential Storage

Sensitive API keys use the **OS keyring** when available. A sentinel value `__KEYRING__` is stored in the JSON config file, and the actual secret is retrieved from the system credential store at runtime.

8.4 Docker Sandbox

The sandbox runs a **Kali Linux** container with pre-installed tools: Nmap, Nuclei, SQLMap, FFuf, Httpx, Katana, Subfinder, Semgrep, Nikto, Gobuster, Arjun, jwt_tool, wafw00f, InteractSH, Playwright/Chromium, Python 3, Node.js, and Go.

Communication occurs via an authenticated HTTP API on port 48081, secured with a per-run `secrets.token_urlsafe()` bearer token.

8.5 Operating Modes

Table 8: Interactive vs. non-interactive modes

Aspect	Interactive (TUI)	Non-Interactive (CLI)
Interface	Rich Live dashboard with panels	Headless, log-only output
Invocation	<code>phantom scan -t <url></code>	<code>phantom scan -t <url> -n</code>
Completion	User can send messages, Ctrl+C	Auto-exits when scan completes
Use Case	Manual pen-testing, exploration	CI/CD pipelines, automation
TTY Required	Yes	No

9 Quality Assurance

9.1 Test Suite

As of v0.9.38:

- **731 tests passing**, 97 skipped (legacy-feature tests)
- Execution time: ~74 seconds on commodity hardware
- Coverage areas: state management, security controls, tool execution, checkpoint/resume, memory compression, cost control, scope validation, report generation, inter-agent sanitisation

9.2 Audit History

Table 9: Audit version history

Version	Key Changes	Tests
v0.9.32	Sandbox startup fix (CRLF, sudoers, port reservation)	~700
v0.9.33	Auto-report pipeline, simplified reporting, phase fallback	~720
v0.9.34	Temperature=None, max_iterations=300, VulnClassTracker disabled	~730
v0.9.35	17 harmful diffs with Strix removed	~735
v0.9.36	CRITICAL: Memory compression bug fix + deep cleanup (−4,050 lines)	737
v0.9.37	Final audit: 9 fixes, dead code, docs, endpoint caps	731
v0.9.38	Zero-base audit: 4 P0, 2 P1, 5 P2 fixes (race condition, auth leak, cost lock)	731

9.3 Benchmark Results

Table 10: OWASP Juice Shop benchmark comparison

Metric	v0.9.32	v0.9.38	Δ
Vulnerabilities found	2	7 (5C + 2H)	+250%
Iterations used	40	26	−35%
Scan time	~5h	15 min	−95%
Cost	~\$2	\$0.36	−82%

10 LLM Selection Guide

10.1 Requirements

Phantom requires models that can:

1. Generate structured XML tool calls reliably
2. Maintain long context (scans produce 50K–150K tokens of history)
3. Reason about multi-step exploit chains
4. Understand HTTP, SQL, JavaScript, and networking concepts
5. Operate at reasonable cost (\$0.10–\$5.00 per scan)

10.2 Model Comparison

Table 11: LLM model comparison for Phantom (35 models across 6 tiers)

#	Model	Context	Cost/M in	Provider	Rating
Tier 1: Recommended (Best Performance)					
1	Claude Sonnet 4	200K	\$3.00	Anthropic	★★★★★
2	GPT-4o	128K	\$2.50	OpenAI	★★★★★

Table 11 — continued

#	Model	Context	Cost/M in	Provider	Rating
3	DeepSeek V3.2	163K	\$0.30	OpenRouter	★★★★★
4	Gemini 2.5 Flash	1M	\$0.15	Google/OpenRouter	★★★★★
5	Qwen3.5-122B	262K	\$0.30	OpenRouter	★★★★★
6	DeepSeek Chat V3-0324	163K	\$0.30	OpenRouter	★★★★★
Tier 2: Strong Alternatives					
7	Grok 4.1 Fast	131K	\$3.00	xAI/OpenRouter	★★★★★
8	Claude 3.5 Haiku	200K	\$0.80	Anthropic	★★★★★
9	GPT-4o Mini	128K	\$0.15	OpenAI	★★★★★
10	Qwen3.5-27B	262K	\$0.27	OpenRouter	★★★★★
11	Qwen3.5-Flash	1M	\$0.10	OpenRouter	★★★★★
12	Llama 3.3 70B	128K	Free	Groq	★★★★★
13	Hermes 3 405B	131K	Free	OpenRouter	★★★★★
14	Gemini 3.1 Flash Lite	1.05M	\$0.25	Google	★★★★★
Tier 3: Budget / Free					
15	Qwen3-Coder (free)	40K	Free	OpenRouter	★★★★★
16	Gemma 3 27B (free)	96K	Free	OpenRouter	★★★★★
17	Mistral Small 3.1 24B	128K	Free	OpenRouter	★★★★★
18	Llama 3.1 8B	128K	Free	Groq	★★★★★
19	Llama 3 70B 8192	8K	Free	Groq	★★★★★
20	DeepSeek V3.1 Terminus	163K	\$0.30	OpenRouter	★★★★★
21	LFM2-24B	33K	\$0.03	Liquid/OpenRouter	★★★★★
22	ByteDance Seed 2.0 Mini	262K	\$0.10	OpenRouter	★★★★★
Tier 4: Self-Hosted / Local					
23	Llama 3:70b (Ollama)	8K	Free	Local	★★★★★
24	Qwen2.5-72B (Ollama)	128K	Free	Local	★★★★★
25	Mistral Large 2 (Ollama)	128K	Free	Local	★★★★★
26	DeepSeek-R1-Distill	128K	Free	Local	★★★★★
27	CodeLlama 70B	16K	Free	Local	★★★★★
Tier 5: Premium / Enterprise					
28	Claude Opus 4	200K	\$15.00	Anthropic	★★★★★
29	GPT-4.5	128K	\$75.00	OpenAI	★★★★★
30	Gemini 2.5 Pro	1M	\$1.25	Google	★★★★★
31	GPT-o3	200K	\$10.00	OpenAI	★★★★★
32	Grok 4.1	131K	\$6.00	xAI	★★★★★
Tier 6: Specialised / Niche					
33	Mistral Codestral	256K	\$0.30	Mistral	★★★★★
34	Cohere Command R+	128K	\$2.50	Cohere	★★★★★
35	Fireworks Llama 3.3 70B	128K	\$0.90	Fireworks AI	★★★★★

10.3 Recommendations

Best Overall

DeepSeek V3.2 via OpenRouter — \$0.30/M input, 163K context, excellent tool-use.
Proven: 7 vulns found at \$0.36/scan.

Best Budget

Llama 3.3 70B via Groq — Free tier, 128K context. Fast inference via Groq hardware. Ideal for development and testing.

Best Premium

Claude Sonnet 4 — 200K context, superior reasoning for complex exploit chains. \$3/M justified for high-value targets.

Best Context Window

Gemini 2.5 Flash — 1M context at \$0.15/M. Ideal for deep scans; the 1M window eliminates most compression needs.

Best Self-Hosted

Qwen2.5-72B via Ollama — 128K context, free, runs on a single A100 GPU. Suitable for air-gapped environments.

11 Recommendations & Future Work

11.1 System Quality Scores

Table 12: System quality assessment (v0.9.38)

Category	Score
Core Scan Engine	95/100
LLM Integration	93/100
Memory Management	95/100
Security Hardening	92/100
Post-Scan Enrichment	98/100
Code Quality	90/100
Documentation	88/100
Test Coverage	87/100
Overall	92/100

11.2 Predictions

- A full 300-iteration scan on Juice Shop should find **15–25 vulnerabilities**
- Estimated cost per full scan: **\$1.50–\$3.00** with DeepSeek V3.2
- With Claude Sonnet 4: expect **20–30 vulnerabilities** at \$8–\$15/scan
- With Gemini 2.5 Flash: expect similar to DeepSeek at **60–80% lower cost**
- v1.0 readiness: 2–3 more development cycles

11.3 Short-Term Roadmap (v0.9.39–v0.9.41)

1. **CI/CD Pipeline** — GitHub Actions for automated testing on push
2. **Benchmark Suite** — Automated scans against DVWA, WebGoat, Juice Shop with regression tracking

3. **SARIF Output** — GitHub Code Scanning integration
4. **Multi-Model Routing** — Cheap model for recon, expensive for exploitation

11.4 Medium-Term (v0.10.x)

1. **Parallel Tool Execution** — Run independent tools concurrently
2. **Agent Collaboration** — Specialised sub-agents (recon, exploit, report)
3. **Web Dashboard** — Real-time scan monitoring via FastAPI + React
4. **Plugin Marketplace** — Community-contributed tools and skills

11.5 Long-Term (v1.0)

1. **Autonomous Red Team Operations** — Multi-target campaign management
2. **Defensive Integration** — WAF bypass testing, IDS evasion assessment
3. **Compliance Automation** — Full PCI DSS / SOC 2 vulnerability mapping
4. **Enterprise Features** — RBAC, multi-tenant, LDAP/SSO integration

11.6 Operator Guidelines

1. Always run Phantom in a controlled environment with explicit authorisation
2. Start with `quick` profile to validate setup before running `deep`
3. Set `PHANTOM_MAX_COST_USD` to prevent unexpected charges
4. Review the audit log (`audit_trail.jsonl`) after every scan
5. Use `--non-interactive` for CI/CD pipelines with `--timeout`
6. Keep the Docker image updated:

```
docker pull ghcr.io/usta0x001/phantom-sandbox:latest
```

12 Conclusion

Phantom v0.9.38 represents a mature, well-audited autonomous security scanning platform that bridges the gap between signature-based vulnerability scanners and human penetration testers. Through 8 rounds of deep audit (v0.9.32–v0.9.38), the system has evolved from a prototype discovering 2 vulnerabilities into a production-capable tool that finds 7+ vulnerabilities in minutes at sub-dollar cost.

The architecture’s strength lies in its clean separation between the reasoning engine (LLM), the action layer (55 tools), and the safety controls (11 defence layers). By aligning the core engine with the proven Strix architecture while retaining genuine Phantom innovations (verification engine, knowledge store, attack graph, compliance mapping), the system achieves both raw scanning power and enterprise-grade reporting.

Key contributions:

- Demonstration that LLM-guided reasoning can match or exceed signature-based scanning for common web vulnerabilities
- A defence-in-depth framework ensuring safe autonomous operation
- A post-scan enrichment pipeline producing compliance-ready reports
- An extensible tool and skill system supporting future growth
- Support for 35+ LLM models across 7 providers with automatic fallback

“The best security tool is the one that thinks like an attacker.”

A Glossary

ReAct Reason + Act — agent paradigm alternating thinking and tool use

CVSS Common Vulnerability Scoring System

IDOR Insecure Direct Object Reference

SSRF Server-Side Request Forgery

XSS Cross-Site Scripting

SQLi SQL Injection

RCE Remote Code Execution

OOB Out-of-Band (callback-based verification)

MITRE ATT&CK Adversary tactics and techniques knowledge base

OWASP Open Web Application Security Project

LiteLLM Python library for unified LLM API access

Jinja2 Python templating engine

PoC Proof of Concept

SSTI Server-Side Template Injection

JWT JSON Web Token

HMAC Hash-based Message Authentication Code

B Full Configuration Reference

See Section 8 for all environment variables. Additional options are available in the `phantom config set` CLI command and the `~/.phantom/cli-config.json` file.